Applications written in cross-platform languages (like Java, C#, Python, etc.) are not binary programs that can run directly on the hardware. Instead, they need the language runtime environment to be present on the OS which interprets the program and runs it. The execution for such a program happens in this manner:

1. When the application is launched, the OS reads the file and determines that the program needs a language runtime to execute. The OS determines this either by reading the file extension or the first few bytes of the file.
2. The OS launches the language environment like any other application and supplies it the original program file it read in step 1. If the runtime is not present, it can show an error or open the file in an editor program.
3. The runtime reads the file. Depending on the language runtime (e.g. Python runtime) it would first interpret the file and generate opcodes (standing for operation code and sometimes called bytecode). Languages like Java *or .NET* expect the input to be in bytecode format.
4. Runtime reads the byte-code and converts it to binary codes which can run on the platform's microprocessor. For performing operations which need support from OS (like reading a file or transmitting data over network), the runtime translates the bytecode instructions into API calls for the OS.
5. Once all execution has been done, the language runtime stops itself and exits.

In these cases, the language runtime must be designed for the platform (Processor architecture and OS).

Software used to design SoCs

So far, we have talked about software that run on SoCs. However, designing one needs a lot of software tools as well.

Design

One of the first things that is required for designing hardware is a language using which various operations of a standard electronic circuit could be described. Such languages are called hardware description languages. VHDL is one of the oldest languages in this realm while Verilog is a newer language with multiple revisions that have kept it relevant and advanced. These languages allow the designer to build the "logical" model of the hardware wherein the expressions of the language define the way circuits would communicate with each other and how electric signals flow among them. High-level synthesis programs are used to write the desired behavior which are converted to their corresponding HDL representations.